

CHƯƠNG 3:

XỬ LÝ THÔNG TIN TRÊN MÁY TÍNH:

TRUY VẤN DỮ LIỆU

Khoa Khoa học và kỹ thuật thông tin
Bộ môn Thiết bị di động và Công nghệ Web

NỘI DUNG

1. Truy vấn SQL.
2. Xpath/Xquery.
3. Các dạng truy vấn select thường gặp.

SQL – Truy vấn SQL

Giới thiệu

- SQL (Structured Query Language):
 - + Là ngôn ngữ cấp cao.
 - + Dùng để truy vấn dữ liệu trong CSDL quan hệ.
 - + Được IBM phát triển (1970s).
 - + Được gọi là SEQUEL.
 - + Được ANSI công nhận và phát triển thành chuẩn SQL-86, SQL-92, SQL-99.
- Đây là ngôn ngữ chuẩn dùng để truy vấn trong các CSDL quan hệ. Các CSDL quan hệ dù khác nhau về nền tảng và hãng sản xuất nhưng luôn có điểm chung là dùng SQL làm ngôn ngữ truy vấn.

Các nhóm lệnh

- Nhóm định nghĩa dữ liệu (DDL - Data Definition Language):
 - + Gồm các lệnh tạo, thay đổi cấu trúc các bảng dữ liệu (*Create, Drop, Alter*)
- Nhóm thao tác dữ liệu (DML - Data Manipulation Language):
 - + Gồm các lệnh làm thay đổi dữ liệu lưu trong bảng (*Insert, Delete, Update, Select*)
- Nhóm điều khiển dữ liệu (DCL - Data Control Language):
 - + Gồm các lệnh quản lý quyền truy cập vào dữ liệu và các bảng (*Grant, Revoke, Deny*)

Phép kết trên nhiều bảng

- Phép ϕ kết, kết tự nhiên
 - + Dùng mệnh đề WHERE chỉ ra điều kiện kết giữa các thuộc tính của các bảng
 - + Hoặc dùng từ khóa **Inner Join** (hoặc Join) trong mệnh đề FROM.
- Phép kết trái, phải, ngoài
 - + Dùng Half Outer Join (**Left join**, **Right Join**), **Full Outer Join** trong mệnh đề FROM

Phép kết trên nhiều bảng (Ví dụ)

NHANVIEN (MANV, HONV, TENLOT, TENNV, NGSINH, DCHI, PHAI, LUONG, MA_NQL, PHG)

PHONGBAN (MAPHG, TENPHG, TRPHG, NG_NC)

DIADIEM (MADIADIEM, TENDIADIEM)

DIADIEM_PHG (MADIADIEM, MAPHG)

THANNHAN (MA_NVIEN, TENTN, PHAI, NGSINH, QUANHE)

DEAN (MADA, TENDA, DDIEM_DA, PHONG)

PHANCONG (MA_NVIEN, MADA, THOIGIAN)

Phép kết trên nhiều bảng (Ví dụ)

NHÂNVIÊN (Mã nhân viên, Họ nhân viên, Tên lót nhân viên, Tên nhân viên, Ngày sinh, Địa chỉ, Giới tính, Lương, Mã người quản lý, Mã phòng) (Với 'Mã nhân viên' là khóa chính, 'Mã phòng' là khóa ngoại đến 'Mã phòng ban' của bảng PHÒNG BAN, 'Mã người quản lý' là khóa ngoại đến 'Trưởng phòng' của bảng PHÒNG BAN)

PHÒNGBAN (Mã phòng ban, Tên phòng, Trưởng phòng, Ngày nhậm chức) (Với 'Mã phòng' là khóa chính)

ĐỊAĐIỂM (Mã địa điểm, Tên địa điểm) (Với 'Mã địa điểm' là khóa chính)

ĐỊAĐIỂM_PHÒNG (Mã địa điểm, Mã phòng ban) (Khóa chính là cả 2 thuộc tính 'Mã địa điểm' và 'Mã phòng ban', 'Mã địa điểm' là khóa ngoại đến 'Mã địa điểm' trong bảng ĐỊA ĐIỂM, 'Mã phòng ban' là khóa ngoại đến 'Mã phòng ban' trong bảng PHÒNG BAN)

THÂN NHÂN (Mã Nvien, Tên thân nhân, Giới tính, Ngày sinh, Quan hệ) (Với 'Mã Nvien' và 'Tên thân nhân' là khóa chính, 'Mã Nvien' là khóa ngoại đến 'Mã nhân viên' trong bảng NHÂN VIÊN)

ĐỀÁN (Mã đề án, Tên đề án, Địa điểm đề án, Phòng ban chủ trì) (Với 'Mã đề án' là khóa chính, 'Phòng ban chủ trì' là khóa ngoại đến 'Mã phòng ban' của bảng PHÒNG BAN)

PHÂN CÔNG (Mã Nvien, Mã đề án, Thời gian) (Với 'Mã Nvien' và 'Mã đề án' là khóa chính, 'Mã Nvien' là khóa ngoại đến 'Mã nhân viên' trong bảng NHÂN VIÊN, 'Mã đề án' là khóa ngoại đến 'Mã đề án' trong bảng ĐỀ ÁN)

Phép kết trên nhiều bảng (Ví dụ)

1. In danh sách mã số, họ tên của tất cả các nhân viên và tên thân nhân của nhân viên đó.

So với

2. In danh sách mã số, họ tên của tất cả các nhân viên và tên thân nhân của nhân viên đó (nếu có).

Vd 1

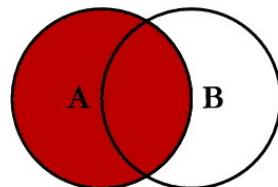
```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten, tentn  
FROM Nhanvien JOIN Thanhnan ON Nhanvien.manv = Thanhnan.ma_nvien
```

Vd 2

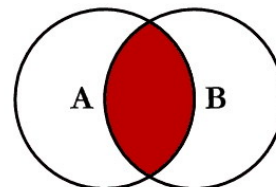
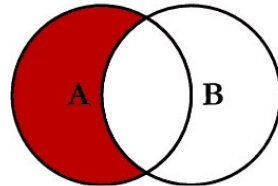
```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten, tentn  
FROM Nhanvien LEFT JOIN Thanhnan ON Nhanvien.manv = Thanhnan.ma_nvien
```

Phép kết trên nhiều bảng

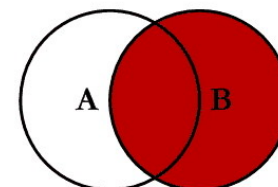
SQL JOINS



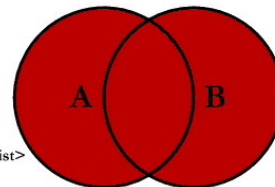
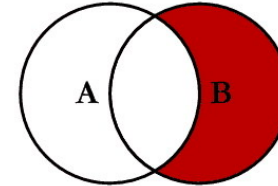
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



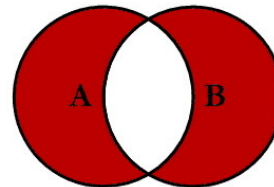
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



© C.L. Moffatt, 2008

TRUY VẤN LÒNG

- Các câu lệnh SELECT có thể lồng nhau ở nhiều mức.
- Các câu truy vấn con thường trả về một tập các giá trị.
- Các câu truy vấn con được kết hợp bằng phép nối logic với câu truy vấn cha.

In và Not In

+ Cú pháp:

<thuộc tính> (NOT) IN (<truy vấn con>)

+ Lưu ý:

- Thuộc tính ở mệnh đề SELECT của truy vấn con phải có cùng kiểu dữ liệu với thuộc tính ở mệnh đề WHERE của truy vấn cha

Ví dụ:

3. Tìm nhân viên chưa được phân công thực hiện đề án nào

```
SELECT manv, honv+' '+tenlot+' '+tennv as hoten
FROM Nhanvien
Where manv NOT IN
      (SELECT distinct ma_nvien FROM Phan cong)
```

Any/Some và All

— Cú pháp:

<thuộc tính> <phép so sánh> Any/Some/All (<truy vấn con>)

— Lưu ý:

- + Thuộc tính ở mệnh đề SELECT của truy vấn con phải có cùng kiểu dữ liệu với thuộc tính ở mệnh đề WHERE của truy vấn cha
- + Any/Some: so sánh với bất kỳ giá trị nào đó trong tập hợp.
- + All: so sánh với tất cả các giá trị trong tập hợp.

VÍ DỤ

Vd 4

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE luong >= ALL
              (SELECT luong FROM Nhanvien)
```

Vd 5

```
SELECT nv1.manv, nv1.honv+'`'+nv1.tenlot+'`'+nv1.tennv as hoten
FROM Nhanvien nv1, Nhanvien nv2
WHERE nv1.phai='Nu' AND nv1.luong>nv2.luong AND nv2.phai='Nam'
```

Vd 6

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE phai= 'Nu' AND luong > ANY
              (SELECT luong FROM Nhanvien Where phai='Nam')
```

NHẬN XÉT

	VD5: Dùng Self JOIN	VD6: Dùng ANY
Ý nghĩa	Lấy nhân viên nữ có lương lớn hơn từng nhân viên nam một	Lấy nhân viên nữ có lương lớn hơn ít nhất một nhân viên nam
Số dòng trả về	Một nữ có thể xuất hiện nhiều lần (tùy số nam thấp hơn)	Mỗi nữ chỉ xuất hiện 1 lần (nếu đáp ứng điều kiện)
Hiệu suất	Chậm hơn khi bảng lớn (JOIN tự kết hợp)	Tốt hơn (dùng subquery)
Trường hợp NULL	Nếu nv2.luong có NULL, kết quả có thể bị sai	Bỏ qua NULL, không gây lỗi

Exists và Not Exists

— Cú pháp:

(NOT) EXISTS (<câu truy vấn con>)

— Lưu ý:

- + Không cần có thuộc tính, hằng số hay biểu thức nào khác đứng trước
- + Không nhất thiết liệt kê tên thuộc tính ở mệnh đề SELECT của truy vấn con
- + Những câu truy vấn có điều kiện “= ANY” hay “IN” đều có thể chuyển thành câu truy vấn có EXISTS

VÍ DỤ

Tìm nhân viên đã được phân công ít nhất một công việc

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten  
FROM Nhanvien  
WHERE EXISTS  
      (SELECT * FROM Phancong WHERE manv=ma_nvien)
```

VÍ DỤ

3. Tìm nhân viên chưa được phân công thực hiện đề án nào

Vd 7

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE manv NOT IN
      (SELECT distinct ma_nvien FROM Phancong)
```

Vd 8

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE manv <> ALL
      (SELECT distinct ma_nvien FROM Phancong)
```

Vd 9

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE NOT EXISTS
      (SELECT * FROM Phancong WHERE manv=ma_nvien)
```

Phân loại TRUY VẤN LÒNG

— Lồng phân cấp:

- + Mệnh đề WHERE của truy vấn con **không tham chiếu** đến thuộc tính của các quan hệ trong mệnh đề FROM ở truy vấn cha
- + Khi thực hiện, câu truy vấn con sẽ được thực hiện trước

— Lồng tương quan:

- + Mệnh đề WHERE của truy vấn con **tham chiếu ít nhất một thuộc tính** của các quan hệ trong mệnh đề FROM ở truy vấn cha
- + Khi thực hiện, câu truy vấn con sẽ được thực hiện nhiều lần, mỗi lần tương ứng với một bộ của truy vấn cha

Lồng phân cấp

Vd 10

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien, Thannhan
WHERE manv=ma_nvien AND quanhe='Con trai'
```

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE manv IN
      (SELECT ma_nvien FROM Thannhan Where quanhe='Con trai')
```

Vd 11

Lồng tương quan

Vd 12

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien, Thannhan
WHERE manv=ma_nvien AND quanhe='Con trai'
```

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE EXISTS
      (SELECT * FROM Thannhan Where manv=ma_nvien AND
                                           quanhe='Con trai')
```

Vd 13

NHẬN XÉT

	Cách hoạt động	Ưu điểm	Nhược điểm	Dùng khi nào
JOIN	Kết hợp hai bảng, lọc quanhe = 'Con trai'	Nhanh với bảng nhỏ, dễ hiểu	Trả về nhiều dòng nếu có nhiều con trai	Khi cần lấy thêm thông tin từ Thanhhan
IN	Lấy danh sách ma_nvien từ Thanhhan, lọc Nhanvien	Dễ hiểu, kết quả không trùng	Chậm hơn với bảng lớn, lỗi nếu có NULL	Khi dữ liệu nhỏ, không có NULL
EXISTS	Kiểm tra tồn tại của dòng trong Thanhhan	Tốt hơn với bảng lớn, không lỗi với NULL	Phức tạp hơn IN	Khi dữ liệu lớn, có NULL

Lồng tương quan

Vd 14

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE manv NOT IN
      (SELECT distinct ma_nvien FROM PHANCONG)
```

```
SELECT manv, honv+'`'+tenlot+'`'+tennv as hoten
FROM Nhanvien
WHERE NOT EXISTS
      (SELECT * FROM Phancong Where manv=ma_nvien)
```



Vd 15

NHẬN XÉT

	IN	NOT IN	EXISTS	NOT EXISTS
Mục đích	Kiểm tra xem một giá trị có thuộc danh sách kết quả từ subquery không	Kiểm tra xem một giá trị có <i>không</i> thuộc danh sách kết quả từ subquery không	Kiểm tra xem có ít nhất một bản ghi nào tồn tại trong subquery không	Kiểm tra xem <i>không</i> có bản ghi nào tồn tại trong subquery không
Cách hoạt động	So sánh từng giá trị với danh sách từ subquery	So sánh từng giá trị với danh sách từ subquery	Dừng ngay khi tìm thấy một kết quả khớp	Dừng ngay khi tìm thấy một kết quả khớp
Hiệu suất với dữ liệu lớn	Chậm hơn nếu danh sách lớn, vì phải kiểm tra tất cả các giá trị	Chậm hơn nếu danh sách lớn, vì phải kiểm tra tất cả các giá trị	Nhanh hơn IN vì chỉ cần tìm thấy một kết quả là dừng	Nhanh hơn NOT IN vì chỉ cần tìm thấy một kết quả là dừng
Khi có NULL	Có thể bị lỗi nếu danh sách chứa NULL	Có thể bị lỗi nếu danh sách chứa NULL	Không bị ảnh hưởng bởi NULL	Không bị ảnh hưởng bởi NULL
Dùng khi nào	Khi danh sách nhỏ và chắc chắn không có NULL	Khi danh sách nhỏ và chắc chắn không có NULL	Khi bảng dữ liệu lớn hoặc có NULL	Khi bảng dữ liệu lớn hoặc có NULL

Một số dạng truy vấn khác

- Câu truy vấn con không chỉ xuất hiện ở mệnh đề WHERE mà có thể xuất hiện ở những nơi khác (SELECT, FROM, HAVING,...)
- Kết quả trả về của câu truy vấn con
 - + Là một bảng trung gian trong quá trình truy vấn
 - + Bảng này không có lưu trữ thật sự

VÍ DỤ

- Vd: Cho biết số lượng nhân viên Nam (phai='Nam'), nhân viên Nữ (phai='Nu') trong từng phòng ban.

Vd 16

```
SELECT MAPHG, TENPHG,  
       (SELECT COUNT(*) FROM NHANVIEN  
        WHERE PHAI='Nam' AND PHG=MAPHG) AS SLNAM,  
       (SELECT COUNT(*) FROM NHANVIEN  
        WHERE PHAI='Nu' AND PHG=MAPHG) AS SLNU  
FROM PHONGBAN
```

- Duyệt từng phòng trong PHONGBAN.
- Với mỗi phòng, chạy 2 subquery riêng biệt để đếm số nhân viên nam (SLNAM) và nữ (SLNU).
- Chạy lại 2 lần subquery cho mỗi dòng → Không tối ưu khi có nhiều phòng ban.

VÍ DỤ

Vd 17

```
SELECT MAPHG, TENPHG, NAM.SLNV, NU.SLNV
FROM PHONGBAN,
    (SELECT PHG, COUNT(*) AS SLNV
     FROM NHANVIEN
     WHERE PHAI='Nam'
     GROUP BY PHG) AS NAM,
    (SELECT PHG, COUNT(*) AS SLNV
     FROM NHANVIEN
     WHERE PHAI='Nu'
     GROUP BY PHG) AS NU
WHERE MAPHG=NAM.PHG AND MAPHG=NU.PHG
```

- Dùng hai subquery NAM và NU trong FROM để tính trước số nhân viên nam và nữ theo từng PHG.
- Kết hợp kết quả với PHONGBAN qua WHERE.
- Tính toán nhóm trước (**GROUP BY PHG**), tránh chạy lại subquery cho từng dòng.

Các phép toán trên tập hợp

— SQL có cài đặt các phép toán

+ Hội (UNION).

+ Giao (INTERSECT).

+ Trừ (EXCEPT).

— Kết quả trả về là tập hợp

+ Loại bỏ các bộ trùng nhau.

+ Để giữ lại các bộ trùng nhau:

- UNION ALL
- INTERSECT ALL
- EXCEPT ALL

Các phép toán trên tập hợp

— SQL có cài đặt các phép toán

+ Hội (UNION)

Lấy các bộ xuất hiện trong một hoặc cả hai truy vấn, nhưng loại bỏ các bộ trùng lặp.

+ Giao (INTERSECT)

Lấy các bộ xuất hiện trong cả hai truy vấn.

+ Trừ (EXCEPT)

Lấy các bộ có trong truy vấn đầu tiên nhưng không có trong truy vấn thứ hai.

Các phép toán trên tập hợp

— Cú pháp:

```
SELECT <các thuộc tính> FROM <các bảng> WHERE <các điều kiện>  
UNION [ALL]  
SELECT <các thuộc tính> FROM <các bảng> WHERE <các điều kiện>
```

```
SELECT < các thuộc tính> FROM <các bảng> WHERE <các điều kiện>  
INTERSECT [ALL]  
SELECT <các thuộc tính> FROM <các bảng> WHERE <các điều kiện>
```

```
SELECT <các thuộc tính> FROM <các bảng> WHERE <các điều kiện>  
EXCEPT [ALL]  
SELECT <các thuộc tính> FROM <các bảng> WHERE <các điều kiện>
```

Các phép toán trên tập hợp

- Tuy nhiên, chúng ta có thể sử dụng **IN**, **NOT IN**, **EXISTS**, **NOT EXISTS**, ... để thực hiện các phép toán hội, giao trừ trên tập hợp.
- Đối với phép chia: sử dụng **NOT EXISTS**.
- Ví dụ:
 - Vd 18: Tìm họ tên nhân viên được phân công thực hiện tất cả các đề án.
 - Vd 19: Tìm tên các đề án được phân công cho tất cả các nhân viên thuộc phòng số 5 thực hiện.

Các phép toán trên tập hợp

Vd 18

```
SELECT manv, honv, tenlot, tennv
FROM Nhanvien
Where NOT EXISTS
      (SELECT * FROM Dean WHERE NOT EXISTS
        (SELECT * FROM Phancong WHERE manv=ma_nvien
          AND Dean.mada=Phancong.mada))
```

Vd 19

```
SELECT tenda
FROM Dean
Where NOT EXISTS
      (SELECT * FROM Nhanvien WHERE phg=5 AND NOT EXISTS
        (SELECT * FROM Phancong WHERE manv=ma_nvien
          AND Dean.mada=Phancong.mada))
```

Hàm kết hợp, gom nhóm

- Hàm kết hợp
- Top N
- Gom nhóm

Hàm kết hợp

— MIN, MAX, SUM, AVG:

TÊN HÀM(<tên thuộc tính>)

— COUNT

- ✓ COUNT(*) đếm số dòng
- ✓ COUNT(<tên thuộc tính>) đếm số dòng thuộc tính có giá trị khác NULL
- ✓ COUNT(DISTINCT <tên thuộc tính>) đếm số dòng thuộc tính có giá trị khác nhau và khác NULL

Ví dụ

- 20. Tính lượng thấp nhất, cao nhất, trung bình và tổng lượng của nhân viên
- 21. Có tất cả bao nhiêu nhân viên
- 22. Có bao nhiêu nhân viên được quản lý trực tiếp bởi người khác
- 23. Có tất cả bao nhiêu người quản lý
- 24. Có bao nhiêu nhân viên không có người quản lý trực tiếp

Vd 20

```
SELECT min(luong) as CN, max(luong) as TN, avg(luong) as TB,
       sum(luong) as TONG
FROM Nhanvien
```

Vd 21

```
SELECT count(*) as SLNV
FROM Nhanvien
```

Vd 22

```
SELECT count(ma_nql) as SLNV
FROM Nhanvien
```

```
SELECT count(*) as SLNV
FROM Nhanvien
WHERE ma_nql IS NOT NULL
```

Vd 23

```
SELECT count(DISTINCT ma_nql) as SLNQL
FROM Nhanvien
```

Vd 24

```
SELECT count(*) as SLNV
FROM Nhanvien
WHERE ma_nql IS NULL
```

Top N

- Trả về N dòng kết quả đầu tiên của câu truy vấn
- Cú pháp: **TOP N**
với N là số nguyên dương
- Nên sử dụng ORDER BY để sắp xếp kết quả

Vd 25

```
SELECT TOP 1 lương as CN  
FROM Nhanvien  
ORDER BY lương DESC
```

Gom nhóm

— Cú pháp:

```
SELECT <các thuộc tính>  
FROM <các bảng>  
[WHERE <các điều kiện>]  
GROUP BY <các thuộc tính gom nhóm>  
[HAVING <các điều kiện>]
```

— Trong đó:

- Điều kiện ở WHERE thực hiện **trước** khi gom nhóm
- Điều kiện ở HAVING thực hiện **sau** khi gom nhóm
- Các thuộc tính sau GROUP BY dùng để gom nhóm và phải có đầy đủ các thuộc tính sau SELECT (trừ những thuộc tính trong những hàm kết hợp)

Ví dụ

- 26. Tìm số lượng nhân viên của từng phòng ban
- 27. Tìm số lượng nhân viên Nam (phai='Nam') của từng phòng ban
- 28. Tìm phòng ban có từ 2 nhân viên Nam (phai='Nam') trở lên
- 29. Tìm phòng ban có đông nhân viên nhất
- 30. Tìm đề án có ít nhân viên Nữ (phai='Nu') tham gia nhất
- 31. Tìm 3 nhân viên thuộc phòng số 4 (phg=4) có lương thấp nhất

Xpath/Xquery

Giới thiệu

— Xpath và Xquery:

- + Là hai ngôn ngữ có rất nhiều mặt giống nhau, hỗ trợ tìm kiếm thông tin trong tài liệu XML.
- + Có thể xem Xpath là tập hợp con của Xquery.
- + Xquery sử dụng Xpath như là một ngôn ngữ chính để định hướng tìm kiếm thay vì dùng đệ quy để duyệt cây.

Xpath

- XML Path language (gọi tắt là Xpath) là một chuẩn để xử lý tài liệu XML (cũng như SQL là một chuẩn để làm việc với csdl).
- Dùng để xử lý nhiều kiểu truy vấn trong tài liệu XML và các biến thể của nó (như HTML).
- Là ngôn ngữ rất phổ biến.
- Tiết kiệm thời gian trích xuất dữ liệu.

Ví dụ 36: CSDL XML

```

1.  <users>
2.    <user>
3.      <name>
4.        <first>Lola</first>
5.        <last>Solis</last>
6.      </name>
7.      <age>2</age>
8.    </user>
9.    <user>
10.     <name>
11.       <first>Nina</first>
12.       <last>Serafina</last>
13.     </name>
14.     <age>4</age>
15.  <visits>
16.    <first>2008-01-15</first>
17.    <last>2008-02-15</last>
18.  </visits>
19. </user>
20. <user>
21.   <name>
22.     <first>Tracy</first>
23.     <last>Keller</last>
24.   </name>
25.   <age>35</age>
26. </user>
27. </users>

```

Ví dụ

— Đoạn Xpath sau: tìm tên của người dùng dưới 18 tuổi

1. `/user[age lt 18]/name/last/text()`
- 2.
3. `(: Result`
4. `Solis`
5. `Serafina`
6. `:)`

— Nếu không dùng Xpath sẽ gặp chút khó khăn khi xử lý việc loại bỏ giá trị trong node visits.

Nhận xét về Xpath

- Biểu thức Xpath ngắn gọn, rõ ràng.
- Hiểu được các node phức tạp trong tài liệu XML và biết các mối quan hệ giữa chúng.
- Một hạn chế của Xpath là *không cung cấp cách chuyển đổi tập kết quả trả về*.
 - + Ở ví dụ trên không thể sắp kết quả hiển thị tăng dần theo tên.

XQuery

- Phức tạp hơn so với Xpath.
- Sử dụng cú pháp pha trộn XML và Xpath.
- Khắc phục được nhược điểm của Xpath:
 - + Sắp xếp kết quả của câu truy vấn hoặc chuyển chúng thành HTML, CSV, SQL, XML ...
 - + Cung cấp tính năng biểu thức FLWOR.
 - + Sử dụng hàm và đệ quy.
 - + Diễn tả các phép nối.

Biểu thức FLOWR

- Biểu thức FLWOR dùng để liên kết các tiêu chí rút trích dữ liệu và chuyển đổi tập kết quả trả về của câu truy vấn.
- FLWOR là viết tắt của các từ **for**, **let**, **where**, **order by** và **return**.
- Bắt đầu bằng một biểu thức *for* hoặc *let* và kết thúc bằng một biểu thức *return*.

Ví dụ 1

— Tìm tên của người dùng dưới 18 tuổi, có sắp xếp kết quả tăng dần.

1. let \$xml:= _XML from **Vidu36**
2. for \$user in \$xml//user[age lt 18]
3. order by \$user/name/last
4. return \$user/name/last/text()

5. (: Result
6. Serafina
7. Solis
8. :)

Ví dụ 2

— Kết quả truy vấn trả về 1 đoạn HTML, danh sách có đánh số thứ tự.

```

1. let $xml:= _XML from Vidu36
2. return
3.     <ol>{
4.         for $user in $xml//user[age lt 18]
5.         order by $user/name/last
6.         return <li>{$user/name/last/text()}</li>
7.     }</ol>
8. (: Result
9.     <ol><li>Serafina</li><li>Solis</li></ol>
10. :)
    
```

Sử dụng hàm và đệ quy

- Xquery cung cấp các hàm, các phép toán được xây dựng sẵn và cho phép định nghĩa các hàm riêng.
- Xquery cũng hỗ trợ đệ quy: tiện lợi khi làm việc với XML (có thể chứa các node lồng nhau tùy ý).

Ví dụ 4

- Định nghĩa hàm transform-names dùng để thay đổi tên các node trong bất kỳ tài liệu XML nào.

Ví dụ 4 – xquery (1)

1. (: Part 1 :)
2. **define function transform-names(\$node as node()) as node() {**
3. **element{replace(name(\$node), "_", "-")} {**
4. **\$node/text(), for \$subnode in \$node/* return transform-names(\$subnode)**
5. **}**
6. **}**

Ví dụ 4 – xquery (2)

```

7. (: Part 2 :)
8. let $xml:=
9. <item>
10.   <item_type>book</item_type>
11.   <contributors>
12.     <author>
13.       <first_name>Charles</first_name>
14.       <last_name>Edward</last_name>
15.       <home_address>
16.         <home_street>206 S. Solomon St.</home_street>
17.         <home_city>New Orleans</home_city>
18.         <home_state>LA</home_state>
19.         <home_zip>70119</home_zip>
20.       </home_address>
21.     </author>
22.     <artist>
23.       <last_name>Salinas</last_name>
24.     </artist>
25.   </contributors>
26. </item>
27. return transform-names($xml)

```

Ví dụ 4 – xquery (3)

```

28. (: Result
29.   <item>
30.     <item-type>book</item-type>
31.     <contributors>

32.       <author>
33.         <first-name>Charles</first-name>
34.         <last-name>Edward</last-name>
35.         <home-address>
36.           <home-street>206 S. Solomon St.</home-street>
37.           <home-city>New Orleans</home-city>
38.           <home-state>LA</home-state>
39.           <home-zip>70119</home-zip>
40.         </home-address>
41.       </author>
42.       <artist>
43.         <last-name>Salinas</last-name>
44.       </artist>
45.     </contributors>
46.   </item>
47. :)
```

MỘT SỐ DẠNG TRUY VẤN SELECT THƯỜNG GẶP

CSDL QUẢN LÝ BÁN HÀNG

KHACHHANG (MAKH, HOTEN, DCHI, SODT, NGSINH, DOANH SO, NGDK)

NHANVIEN (MANV, HOTEN, NGVL, SODT)

SANPHAM (MASP, TENS P, DVT, NUOCSX, GIA)

HOADON (SOHD, NGHD, MAKH, MANV, TRIGIA)

CTHD (SOHD, MASP, SL)

TRUY VẤN SELECT

— Cú pháp câu truy vấn *SELECT*:

SELECT <cột 1>, <cột 2>,

FROM <tên bảng>

WHERE <điều kiện>

ORDER BY <tên cột> *ASC* | *DESC*

GROUP BY <tên cột 1>, <tên cột 2>,

HAVING <điều kiện>

— Lưu ý:

+ Mệnh đề *HAVING* sử dụng cho các hàm gom nhóm.

+ *ASC* – sắp xếp tăng dần; *DESC* – sắp xếp giảm dần.

TRUY VẤN SELECT – DẠNG 1

– **Dạng 1:** Truy vấn lấy dữ liệu tất cả.

```
SELECT * FROM <tên bảng>
```

hoặc

```
SELECT <danh sách cột> FROM <tên bảng>
```

VD:

```
SELECT * FROM KHACHHANG
```

```
SELECT MAKH, HOTEN, DCHI FROM KHACHHANG
```

TRUY VẤN SELECT – DẠNG 2

– **Dạng 2:** Truy vấn dữ liệu có điều kiện:

```
SELECT <danh sách cột> FROM <tên bảng>  
WHERE <điều kiện>
```

VD: Lấy thông tin MAKH, HOTEN DCHI của khách hàng có doanh số trên 1000

```
SELECT MAKH, HOTEN, DCHI FROM KHACHHANG  
WHERE DOANHISO > 10000
```

TRUY VẤN SELECT – DẠNG 3

– **Dạng 3:** Truy vấn dữ liệu có kết bảng.

SELECT <danh sách cột> FROM <tên bảng 1>

INNER JOIN <tên bảng 2> ON <tên bảng 1>.<mã khoá
ngoại> = <tên bảng 2>.<mã khoá chính>

[WHERE <điều kiện>]

– Các phép kết:

+ **INNER JOIN:** kết bằng.

+ **LEFT OUTER JOIN:** kết mở rộng về bên trái.

+ **RIGHT OUTER JOIN:** kết mở rộng về bên phải.

VD

VD: Tìm danh sách khách hàng đã mua hàng vào ngày 16/7/2019.

```
SELECT MAKH, HOTEN  
FROM HOADON INNER JOIN KHACHHANG ON KHACHHANG.MAKH  
= HOADON.MAKH  
WHERE NGHD = '16/7/2019'
```

TRUY VẤN SELECT – DẠNG 4

– **Dạng 4:** Truy vấn dữ liệu có sắp xếp

```
SELECT <danh sách tên cột> FROM <tên bảng>  
[WHERE <điều kiện>]
```

```
ORDER BY <danh sách cột cần sắp xếp> ASC hoặc DESC
```

VD: Sắp xếp khách hàng theo ngày sinh giảm dần

```
SELECT MAKH, HOTEN, NGSINH FROM KHACHHANG  
ORDER BY NGSINH DESC
```

TRUY VẤN SELECT – DẠNG 5

- **Dạng 5:** Truy vấn sử dụng các hàm gom nhóm

SELECT <các hàm gom nhóm> **FROM** <tên bảng>
[WHERE <điều kiện>]

GROUP BY <tên cột 1>, <tên cột 2>, ...

- Các hàm gom nhóm: *COUNT()*, *AVG()*, *MAX()*, *MIN()*, *SUM()*.

- **Lưu ý:** Các thuộc tính trong mệnh đề **SELECT** (trừ các hàm kết hợp), phải xuất hiện trong mệnh đề **GROUP BY**.

VD

VD: Tính giá trị trung bình doanh số theo từng MAKH đối với khách hàng có doanh số trên 10000

```
SELECT MAKH, AVG(DOANHSON) FROM KHACHHANG  
WHERE DOANHSON > 10000  
GROUP BY MAKH
```

TRUY VẤN SELECT – DẠNG 6

– **Dạng 6:** Truy vấn sử dụng hội - giao - trừ

```
SELECT <danhsách cột 1> FROM <tên bảng>  
[WHERE <điều kiện 1>]
```

UNION (hội) | INTERSECT (giao) | EXCEPT (trừ)

```
SELECT <danhsách cột 2> FROM <tên bảng>  
[WHERE <điều kiện 2>]
```

Lưu ý: Để sử dụng các phép hội giao trừ thì 2 quan hệ phải **khả hợp**,
tức *<danhsách cột 1> = <danhsách cột 2>*

VD

VD1: Tìm khách hàng mua hoá đơn HD01 hoặc HD02

```
SELECT MAKH, HOTEN FROM KHACHHANG INNER JOIN  
HOADON ON KHACHHANG.MAKH = HOADON.MAKH
```

```
WHERE MAHD = 'HD01'
```

UNION

```
SELECT MAKH, HOTEN FROM KHACHHANG INNER JOIN  
HOADON ON KHACHHANG.MAKH = HOADON.MAKH
```

```
WHERE MAHD = 'HD02'
```

VD

VD2: Tìm khách hàng mua cùng lúc hoá đơn HD01 và HD02

```
SELECT MAKH, HOTEN FROM KHACHHANG INNER JOIN  
HOADON ON KHACHHANG.MAKH = HOADON.MAKH
```

```
WHERE MAHD = 'HD01'
```

INTERSECT

```
SELECT MAKH, HOTEN FROM KHACHHANG INNER JOIN  
HOADON ON KHACHHANG.MAKH = HOADON.MAKH
```

```
WHERE MAHD = 'HD02'
```

VD

VD3: Tìm khách hàng không mua hoá đơn nào

```
SELECT MAKH, HOTEN FROM KHACHHANG
```

```
EXCEPT
```

```
SELECT MAKH, HOTEN FROM KHACHHANG INNER JOIN  
HOADON ON KHACHHANG.MAKH = HOADON.MAKH
```

TRUY VẤN SELECT – DẠNG 7

– Dạng 7: Truy vấn lồng:

```
SELECT <danh sách cột> FROM <tên bảng>  
WHERE <so sánh tập hợp> (  
    SELECT <danh sách cột> FROM <tên bảng>  
    WHERE <điều kiện>  
)
```

<so sánh tập hợp>: *ALL, IN, NOT IN, ALL, ANY, EXISTS, NOT EXISTS.*

VD

VD: Tìm thông tin mã hoá đơn có trị giá cao nhất

```
SELECT SOHD FROM HOADON
```

```
WHERE TRIGIA = (
```

```
    SELECT MAX(TRIGIA) FROM HOADON
```

```
)
```

TRUY VẤN SELECT – DẠNG 8

– Dạng 8: Truy vấn lồng tương quan

```
SELECT <danh sách cột> FROM <tên bảng> AS OB1  
WHERE <so sánh tập hợp> (  
    SELECT <danh sách cột> FROM <tên bảng> AS OB2  
    WHERE OB1.<tên cột> = OB2.<tên cột>  
)
```

<so sánh tập hợp>: *ALL, IN, NOT IN, ALL, ANY, EXISTS, NOT EXISTS.*

VD

VD: Tìm tất cả các nước sản xuất có giá cao nhất của từng sản phẩm.

```
SELECT MASP, NUOCSX, GIA
FROM SANPHAM AS SP1
WHERE GIA = (
    SELECT MAX(GIA) FROM SANPHAM AS SP2
    WHERE SP1.MASP = SP2.MASP
)
```

TRUY VẤN SELECT – DẠNG 9

– Dạng 9: Truy vấn dùng bảng “con” (inner aggregate).

```
SELECT <danh sách cột 1> FROM (  
    SELECT <danh sách cột 2> FROM <tên bảng>  
    WHERE <điều kiện>  
) AS <tên bảng con>
```

Lưu ý: <danh sách cột 1> phụ thuộc vào <danh sách cột 2> trả về từ câu truy vấn con (subquery).

VD

VD: Tìm khách hàng có số lần mua hàng nhiều nhất.

```
SELECT MAKH FROM HOADON
GROUP BY MAKH
HAVING COUNT(SOHD) = (
    SELECT MAX(SL_HD) FROM (
        SELECT MAKH, COUNT(SOHD) AS SL_HD
        FROM HOADON
        GROUP BY MAKH) AS T
    )
```

Lưu ý: Không thể sử dụng dạng *COUNT (MAX (SOHD))* trong SQL Server

TRUY VẤN SELECT – DẠNG 10

– Dạng 10: Phép chia

Tìm *<đối tượng 1>* đã ... tất cả *<đối tượng 2>*

Cần xác định:

Đối tượng 1 (MaDT1,).

Đối tượng 2 (MaDT2,).

Quan hệ Đối tượng 1 và đối tượng 2 (*MaDT1*, *MaDT2*,).

VD

```

SELECT <danh sách cột> FROM <tên bảng đối tượng 1>
AS OB1 WHERE NOT EXISTS (
    SELECT <danh sách cột> FROM <tên bảng đối
    tượng 2> AS OB2 WHERE AND NOT EXISTS (
        SELECT * FROM <tên bảng quan hệ đối tượng
        1 và 2> as OB3 WHERE OB2.<khoá chính> = OB3.<khoá
        ngoại> and OB3.<khoá ngoại> = OB1.<khoá chính>
    )
)

```

VD

— VD: Tìm hoá đơn đã mua tất cả sản phẩm xuất xứ Thái Lan:

Đối tượng 1: HOADON(SOHD, NGHD, ...)

Đối tượng 2: SANPHAM(MASP, TENSP, XUATXU)

Quan hệ giữa 2 đối tượng: CTHD(MASP, TENSP).

```
SELECT SOHD FROM HOADON AS T1 WHERE NOT EXISTS (
    SELECT MASP FROM SANPHAM AS T2 WHERE XUATXU =
    "Thái Lan" AND NOT EXISTS (
        SELECT MASP, TENSP FROM CTHD AS T3 WHERE
        T2.MASP = T3.MASP AND T1.SOHD = T3.SOHD)
    )
```

Tổng kết

1. Để truy vấn dữ liệu trong CSDL quan hệ, ta dùng ngôn ngữ SQL.
2. SQL có 3 nhóm lệnh chính: DDL, DML và DCL.
3. Các truy vấn nâng cao trên SQL: kết (nhiều bảng), gom nhóm, truy vấn lồng.
4. Xpath/Xquery là ngôn ngữ dùng để truy vấn CSDL XML.
5. Xquery là phiên bản mở rộng của Xpath, hỗ trợ các hàm, đệ quy, ... mà bản thân Xpath không làm được.

TÀI LIỆU THAM KHẢO

1. Nguyễn Gia Tuấn Anh, Trương Châu Long, *Bài tập và bài giải SQL Server*, NXB Thanh niên (2005).
2. Đỗ Phúc, Nguyễn Đăng Ty, *Cơ sở dữ liệu*, NXB Đại học quốc gia TP HCM (2010).
3. Nguyễn Gia Tuấn Anh, Mai Văn Cường, Bùi Danh Hường, *Cơ sở dữ liệu nâng cao*, NXB Đại học quốc gia TP HCM (2019).
4. Itzik Ben-Gan, *Microsoft SQL Server 2012- TSQL Fundamentals*.

